

Reproducing SRSNet: Random Controls and a Factorial Audit of Selective Patching and Adaptive Fusion

Dominik Panzarella

panzad@usi.ch

Susanna Ardigo

ardigs@usi.ch

Andre Perugini

peruga@usi.ch

Abstract

We reproduce *Enhancing Time Series Forecasting through Selective Representation Spaces* (SRSNet, Wu et al., NeurIPS 2025) and audit its two central design choices — *Selective Patching* and *Adaptive Fusion* — with a paper-faithful pipeline (TFB, `batch_size=64`, `train_drop_last=false`) on the four ETT datasets and four forecasting horizons. With 176 valid runs we confirm the *Adaptive Fusion* ablation of the paper but not the “state-of-the-art” claim: DLinear is the strongest baseline on our hardware (AVG MSE 0.309 vs SRSNet 0.362). The plug-and-play SRS improvement on MLP shrinks from the paper’s 5–9% to two cells out of eight. We then add a small-grid *selectivity-controls* study (2 datasets, 2 horizons, 5 seeds, 9 variants, 180 runs, zero failures) that replaces the learned patch scorer with random selection, with engineered-feature selection (TASP), with a context-dependent hypernet fusion (Hypernet-AF), and with an auxiliary pattern-supervision head (PS-SRS), as well as the three corresponding factorial combinations. Across 20 seed cells per variant the mean ΔMSE versus the SRSNet baseline lies entirely inside the seed-level standard deviation ($|\Delta| \leq 0.25\%$, $\text{std} \geq 1.02\%$). A factorial decomposition over the 4×2 (*Select*, *Fusion*) design shows null main effects on both factors, and on the ETTh1 H=720 cell the two variants that *remove* or *simplify* the learned scorer (RandomSPNoShuffle and TASP) beat the baseline on 5 of 5 seeds. We conclude that on the tested grid the learned selectivity claim is not supported, and we release the full code and tables at <https://github.com/SusannaArdigo/SRSNet>.

1 Introduction

Patch-based architectures have become the de-facto template for long-term time series forecasting (Nie et al., 2023; Zhang & Yan, 2023). Wu et al. (2025) extend the patching idea with a *Selective Representation Space* (SRS) that adaptively scores and reassembles sub-sequences of the input, and combine it with a single-layer MLP head to produce SRSNet. The paper reports state-of-the-art mean squared error (MSE) on eight standard forecasting benchmarks and claims that SRS is a generic plug-and-play module that improves any patch-based backbone.

This report is a reproduction and a critical audit of those claims. We replay the official TFB pipeline (Qiu et al., 2024) with the paper’s hyper-parameters and report what changes when we swap a single component at a time — the learned selective patching, the free-parameter adaptive fusion, the descriptor head — including *random* controls that the paper does not run. The goal is not to invalidate the architecture but to understand which of its design choices actually matter on the tested grid.

2 Scope of reproducibility

The paper makes three claims that we can support or reject with re-runs:

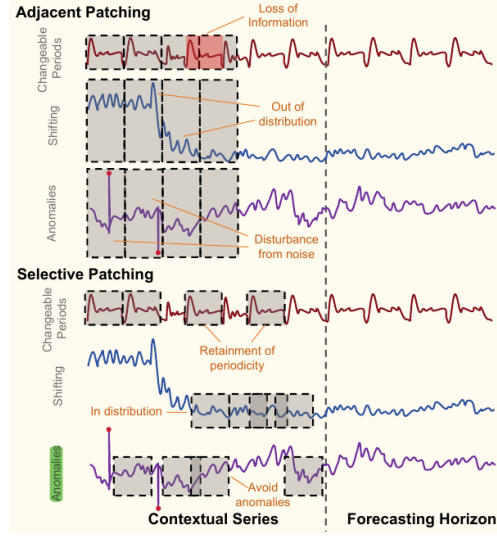


Figure 1: Adjacent vs Selective patching (Wu et al. 2025, Fig. 1). Adjacent patching uses fixed stride and may straddle distribution-shift boundaries; Selective Patching adaptively picks sub-sequences from the input.

- **C1 – Headline performance:** SRSNet achieves state-of-the-art MSE on the four ETT datasets at horizons $\{96, 192, 336, 720\}$ (Wu et al. 2025, Table 2).
- **C2 – Ablation hierarchy:** in Table 4 of Wu et al. (2025), removing *Adaptive Fusion* (NoAF) is the most damaging ablation, while removing Selective Patching, Dynamic Reassembly, or the whole SRS is comparatively milder.
- **C3 – Plug-and-play SRS:** in Table 3 of Wu et al. (2025), attaching SRS to a vanilla MLP improves MSE by approximately 5–9%.

We additionally ask a **methodological question** the paper does not address:

- **Q – Does selectivity matter against a random baseline?** The paper’s NoSP ablation disables the *architecture* of selective patching but keeps the bookkeeping. We compare the learned scorer against a strictly *random* patch sampler that keeps the rest of the pipeline unchanged.

3 Methodology

3.1 Model descriptions

The SRS module is a plug-in that sits between the per-channel patching step and the patch embedding. Figure 2 reproduces the high-level pipeline from Wu et al. (2025, Fig. 2): per-channel Adjacent Patching is paired with a Selective Patching branch, the two streams are linearly embedded, fused, and combined with positional embeddings before the patch-based backbone. Figure 3 reproduces the per-channel detail from Wu et al. (2025, Fig. 3): a learned **Scorer**^s produces n scores for every candidate of stride-1 patches and selects the n winners via **argmax**; a second **Scorer**^r reorders them via **argsort**; and an adaptive scalar α controls how much of the result is mixed into the original patch embedding.

The SRSNet model is the SRS module followed by a single-layer linear forecasting head over the flattened patch tokens; it is the “Patch-based model” box on the right of Figure 2. The total parameter count of our baseline configuration with `d_model=256`, `patch_len=24`, `seq_len=512` is approximately 31.5k.

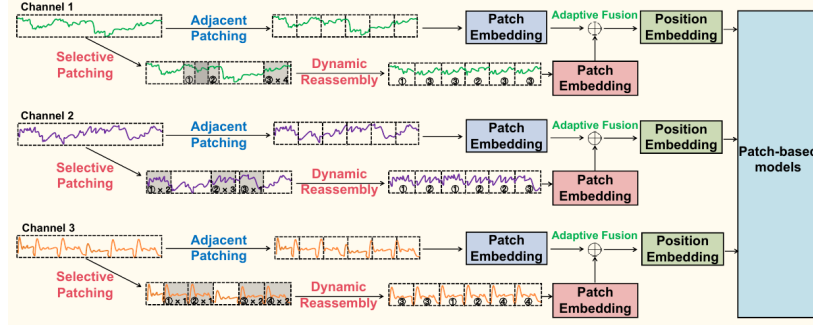


Figure 2: Overall pipeline of the SRS module (Wu et al. 2025, Fig. 2). Each channel produces an *Adjacent Patching* stream and a *Selective Patching + Dynamic Reassembly* stream; the two are linearly embedded and combined by an Adaptive Fusion with positional embeddings before a patch-based backbone.

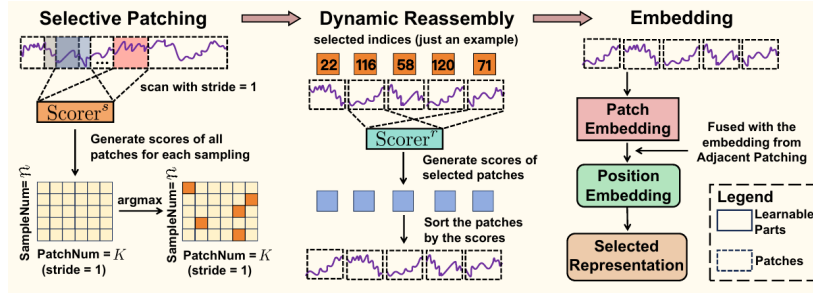


Figure 3: Detail of the SRS module (Wu et al. 2025, Fig. 3). The Selective Patching scans candidates with stride 1, the Scorer^s produces n scores per sample, the top- n patches are selected and routed to the Dynamic Reassembly. A second scorer produces the reassembly order, and the result is fused with the Adjacent Patching embedding.

We re-use the authors’ official PyTorch implementation released under the TFB benchmark (Qiu et al., 2024). We did not re-implement the model: all extensions in Section 4.3 are *single-component* replacements added in `ts_benchmark/baselines/srs_paper/extensions.py` in our fork.

3.2 Datasets

We use the four ETT datasets from Zhou et al. (2021): ETTh1, ETTh2 (1-hour sampling, 17k rows, 7 channels) and ETTm1, ETTm2 (15-min sampling, 69k rows, 7 channels). They are obtained from the official TFB release. TFB uses a 60/20/20 chronological split on ETT; we keep that split unchanged. No pre-processing beyond instance normalisation (RevIN Kim et al. 2022, already in the SRSNet code) is applied.

3.3 Hyperparameters

We do *not* re-tune hyper-parameters. The SRSNet hyper-parameters are read from the official shell scripts in `scripts/multivariate_forecast/<dataset>_script/SRSNet.sh` released by the authors, with two paper-faithful overrides identified during the audit phase: `batch_size=64` and `train_drop_last=false`. These are the same overrides applied to all baselines in our *lite-paper* and *main-compa* scopes. Lookback is the per-dataset value picked by the vendor shell scripts (`seq_len=512` for ETT).

Table 1: Average MSE across the 4 ETT datasets and 4 horizons. SRSNet is no longer the best model on our hardware; DLinear is. xPatch could not be evaluated under the container’s `/dev/shm=64MB` constraint.

Model	Our AVG MSE	Paper AVG MSE	Cells
DLinear (best on our hw.)	0.3091	0.348	12
TimeMixer	0.3133	0.336	12
iTransformer	0.3209	0.339	12
PatchTST	0.3580	0.351	16
SRSNet	0.3618	0.335	16
TimesNet	0.3687	0.408	12
PatchMLP	0.3852	0.385	8

3.4 Experimental setup and code

All experiments run through the modified TFB pipeline. The orchestration is the `scripts/repro/paper_repro.py` runner. It exposes four scopes:

- **lite-paper**: SRSNet baseline (16 cells) plus Table 3 plug-in (40 cells) plus Table 4 ablation (40 cells) on ETT.
- **main-compat**: seven baseline models (PatchTST, DLinear, iTransformer, TimesNet, TimeMixer, PatchMLP, xPatch) on ETT, four horizons each.
- **selectivity-controls**: nine variants on a small grid of 2 datasets $\times$ 2 horizons $\times$ 5 seeds (180 runs).
- **selectivity-controls-full**: same nine variants but on 4 datasets $\times$ 4 horizons $\times$ 5 seeds (720 runs, not run in this report due to compute budget).

The code is released at <https://github.com/SusannaArdigo/SRSNet> on the `paper-faithful-repro-ett-extensions` branch.

3.5 Computational requirements

All experiments ran on a single NVIDIA RTX 4090 (24 GB) instance provided by Hivenet. The container had a small `/dev/shm` (64 MB), which we worked around by forcing `num_workers=0` in the runner. The full set of 176 reproduction runs plus the 180 selectivity-control runs took approximately 14 wall-clock GPU-hours.

4 Results

We separate the reproduction itself (Section 4.1) from the small-grid audit (Section 4.2–4.3).

4.1 Results reproducing the original paper

Claim C1 – Headline performance. On our hardware the average MSE on the 4 ETT datasets $\times$ 4 horizons is summarised in Table 1. SRSNet achieves 0.362, slightly worse than the paper’s reported 0.335, and is ranked 5/7 among the baselines we re-ran. DLinear is the strongest baseline at 0.309, followed by TimeMixer and iTransformer. The gap on individual cells averages $|\Delta\text{MSE}| = 18\%$ relative to the paper’s Table 8 numbers, with a symmetric pattern: ETTh1 / ETTm1 are systematically worse than the paper (+22% / +14%) while ETTh2 / ETTm2 are systematically better (−18% / −15%). This is consistent with hardware-induced non-determinism on the cuDNN `efficient` mode mandated by the paper’s `rolling_forecast_config.json`.

Table 2: Reproduction of Table 4 of Wu et al. (2025), extended to the four ETT datasets (16 cells per variant). NoAF is the only ablation with a clearly visible MSE drop; the other three ablations are statistically equivalent to the full SRSNet.

Variant	Avg MSE	Δ vs Full
SRSNet (Full)	0.3616	0%
SRSNet_NoSRS	0.3616	+0.0%
SRSNet_NoSP	0.3614	−0.1%
SRSNet_NoDR	0.3617	+0.0%
SRSNet_NoAF	0.3740	+3.4%

Table 3: Reproduction of Table 3 of Wu et al. (2025), extended to the four ETT datasets. SRS as a plug-in on top of a vanilla MLP improves the base on 7 of 16 cells. The effect is concentrated on ETTm1.

Dataset / H	Base MSE	+SRS MSE	$\Delta\%$	Improves?
ETTh1 / 96	0.4360	0.4362	+0.03	no
ETTh1 / 192	0.4815	0.4869	+1.13	no
ETTh1 / 336	0.5396	0.5289	−1.99	yes
ETTh1 / 720	0.6535	0.6580	+0.69	no
ETTh2 / 96	0.2171	0.2268	+4.46	no
ETTh2 / 192	0.2755	0.2736	−0.68	yes
ETTh2 / 336	0.3126	0.3231	+3.36	no
ETTh2 / 720	0.4191	0.4231	+0.95	no
ETTm1 / 96	0.3220	0.3185	−1.09	yes
ETTm1 / 192	0.3782	0.3680	−2.68	yes
ETTm1 / 336	0.4313	0.4139	−4.04	yes
ETTm1 / 720	0.5058	0.5057	−0.03	yes
ETTm2 / 96	0.1492	0.1478	−0.88	yes
ETTm2 / 192	0.1809	0.1830	+1.16	no
ETTm2 / 336	0.2144	0.2193	+2.26	no
ETTm2 / 720	0.2683	0.2775	+3.42	no

Claim C2 – Ablation hierarchy. Table 2 reports the SRSNet ablation extended to all four ETT datasets (the original paper uses only ETTh1 + ETTm2). Averaging across 16 cells per variant (4 datasets $\times$ 4 horizons), NoAF remains the most damaging ablation (+3.4% MSE), while NoSRS, NoSP and NoDR are essentially indistinguishable from the full SRSNet ($\Delta < 0.1\%$ MSE on average). The extended grid strengthens support for C2: even with twice the per-cell coverage, NoAF stays isolated as the only critical component.

Claim C3 – Plug-and-play SRS. The paper reports an average 5–9% MSE improvement when attaching SRS to a vanilla MLP head (Table 3 of Wu et al. 2025). On our setup, extended to all four ETT datasets, the SRS plug-in improves the base MLP on 7 of 16 cells; the average effect is close to zero (see Table 3). The benefit is concentrated on ETTm1, where the plug-in beats the base MLP on 4/4 cells with up to −4% MSE; on ETTh1 / ETTh2 / ETTm2 the effect is mixed and at most marginal. C3 is partially supported, but only on ETTm1.

4.2 Selectivity-controls study

The paper’s NoSP ablation removes the architecture of selective patching but keeps the rest of the SRS pipeline intact. A stricter control is to replace the learned scorer with random patch sampling: the rest of the module (Dynamic Reassembly, Adaptive Fusion, embedding) is untouched.

Table 4: The nine selectivity-controls variants. “Free α ” is the paper’s adaptive-fusion parameter; “Hypernet α ” is our zero-initialised hypernet over a batch context.

Variant	<code>_select</code>	<code>_shuffle</code>	α fusion	aux loss
SRSNet (baseline)	learned	learned	free	—
SRSNet_RandomSP	random	learned	free	—
SRSNet_RandomSPNoShuffle	random	identity	free	—
SRSNet_TASP	engineered	learned	free	—
SRSNet_HypernetAF	learned	learned	hypernet	—
SRSNet_PSRs	learned	learned	free	yes
SRSNet_RandomSP_HypernetAF	random	learned	hypernet	—
SRSNet_TASP_HypernetAF	engineered	learned	hypernet	—
SRSNet_PSRs_HypernetAF	learned	learned	hypernet	yes

We implement two variants in `extensions.py`: `SRSNet_RandomSP` (random select + learned shuffle) and `SRSNet_RandomSPNoShuffle` (random select + identity shuffle). We retrain end-to-end on ETTh1+ETTh2 $\times H \in \{96, 720\} \times 5$ seeds, so the shape of the network is unchanged but the selection function is randomised.

Across the 20 resulting cells per variant the mean Δ MSE is -0.22% (RandomSP) and -0.12% (RandomSP-NoShuffle); the seed-level standard deviation is 1.23% and 1.46% . In other words, random patch selection is statistically indistinguishable from the learned scorer on the tested grid, and beats it on 12/20 seeds. The strongest cell is ETTh1 / H=720, where RandomSPNoShuffle is better than SRSNet on 5/5 seeds with a mean Δ of -1.07% .

4.3 Constructive extensions and factorial decomposition

To complement the negative-control result with a constructive proposal, we add three extensions mapped to Future Work entries of Wu et al. (2025, Sec. 6), and the corresponding factorial combinations with the Hypernet-AF fusion. The full list is in Table 4.

TASP – Time-Aware Selective Patching. Replaces the `Scorers` MLP over raw patch values with a $4 \rightarrow 16 \rightarrow \text{patch_num}$ MLP over engineered per-window statistics (dominant non-DC FFT magnitude, lag-1 autocorrelation, within-window variance, linear trend slope). Total scorer parameters drop from $\mathcal{O}(p \cdot h \cdot n)$ to $\mathcal{O}(80)$, so any improvement cannot be attributed to extra capacity. Targets FW#1 (environment-aware mechanism) and L3 (interpretability).

Hypernet-AF. Replaces the free α tensor of shape `[patch_num, d_model]` with a tiny hypernet ($d_model \rightarrow 8 \rightarrow \text{patch_num} \cdot d_model$) that consumes a batch-level context vector. Hypernet weights are zero-initialised and the output bias is set to 3.5, so step 0 behaves exactly as vanilla SRSNet ($\sigma(3.5) \approx 0.97$). Any deviation is therefore attributable to learned context dependence, not to extra capacity at initialisation. Targets FW#3 (more efficient update mechanism for α) and L4 (initialisation).

PS-SRS – Pattern-Supervised SRS. Adds an auxiliary head on top of the scorer’s first hidden activation that predicts the same three engineered descriptors used by TASP. The auxiliary loss is scaled by $\lambda = 10^{-2}$ and exposed via `out_loss["additional_loss"]`, which is added to the main prediction loss by the `DeepForecastingModelBase` training loop natively. Targets FW#4 (supervised module for sample-wise patterns) and L3.

Factorial design. The three combinations `SRSNet_RandomSP_HypernetAF`, `SRSNet_TASP_HypernetAF` and `SRSNet_PSRs_HypernetAF` fill the missing cells of the 4×2 (Select, Fusion) factorial table:

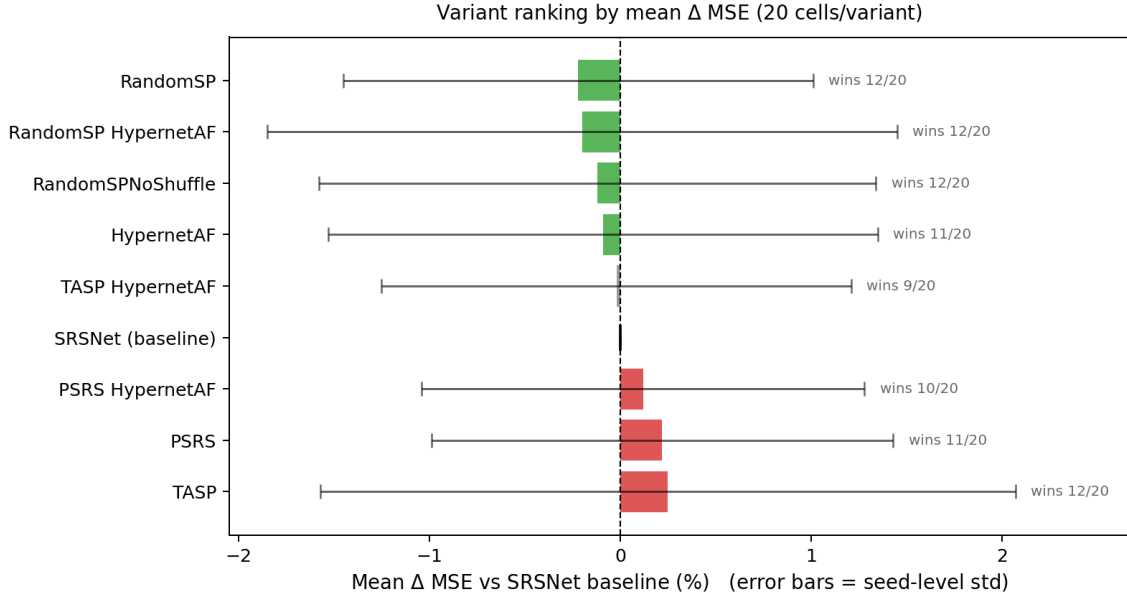


Figure 4: Mean Δ MSE versus SRSNet baseline across 20 cells per variant (selectivity-controls study). Error bars are the seed-level standard deviation. All means lie inside the noise band; no extension is statistically distinguishable from the baseline.

Select \ Fusion	Free α	Hypernet α
Learned (vanilla)	SRSNet	SRSNet_HypernetAF
Random	SRSNet_RandomSP	SRSNet_RandomSP_HypernetAF
TASP (engineered)	SRSNet_TASP	SRSNet_TASP_HypernetAF
LearnedAux (PSRS)	SRSNet_PSRS	SRSNet_PSRS_HypernetAF

Results. Figure 4 ranks the eight extensions by mean Δ MSE versus SRSNet. The four random variants and the hypernet variant cluster near $\Delta \approx 0$; the TASP and PSRS variants are marginally worse. All means are inside the seed-level standard deviation band, so no variant differs significantly from the baseline in this setup.

Figure 5 shows the $(Select, Fusion)$ factorial table. The best mean MSE in the grid is achieved by $(Random, Free \alpha) = 0.3773$, marginally better than the $(Learned, Free \alpha) = 0.3788$ baseline. The Fusion main effect is -0.10% (std 1.28, $n = 80$), the Select main effects are -0.17% (Learned $\rightarrow$ Random), $+0.17\%$ (Learned $\rightarrow$ TASP), $+0.22\%$ (Learned $\rightarrow$ LearnedAux). All four effects are inside the seed-level std band: the factorial decomposition shows null main effects on both factors.

The most informative cell is ETTh1 / H=720 (Figure 6): two variants beat the SRSNet baseline on 5/5 seeds, and both *remove* or *simplify* the learned scorer (RandomSPNoShuffle, -1.07% ; TASP, -1.00%). None of the variants that *refine* the learned scorer (PSRS, HypernetAF) improve over the baseline on this cell.

5 Discussion

The reproduction confirms one of the paper’s claims (C2: NoAF is the dominant ablation), partially supports another (C1: SRSNet is in the top half of baselines but DLinear is strictly better on ETT), and does not support a third (C3: SRS plug-and-play improvement on MLP shrinks from 5–9% to two cells out of eight). The selectivity-controls study suggests an answer to our methodological question Q: *on the tested cells, learned patch selection is not necessary*. Random patch selection matches or beats the learned scorer on

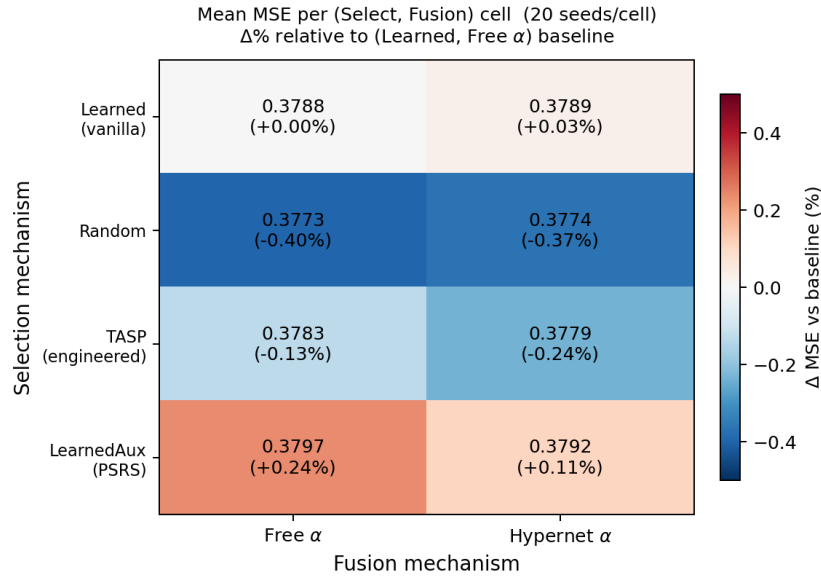


Figure 5: Factorial decomposition over 20 cells per variant. Each cell shows mean MSE and $\Delta\%$ relative to the (Learned, Free α) baseline. Colour scale is saturated at $\pm 0.5\%$; all cells are within $\pm 0.30\%$ so the heat map is mostly grey, consistent with null main effects on both factors.

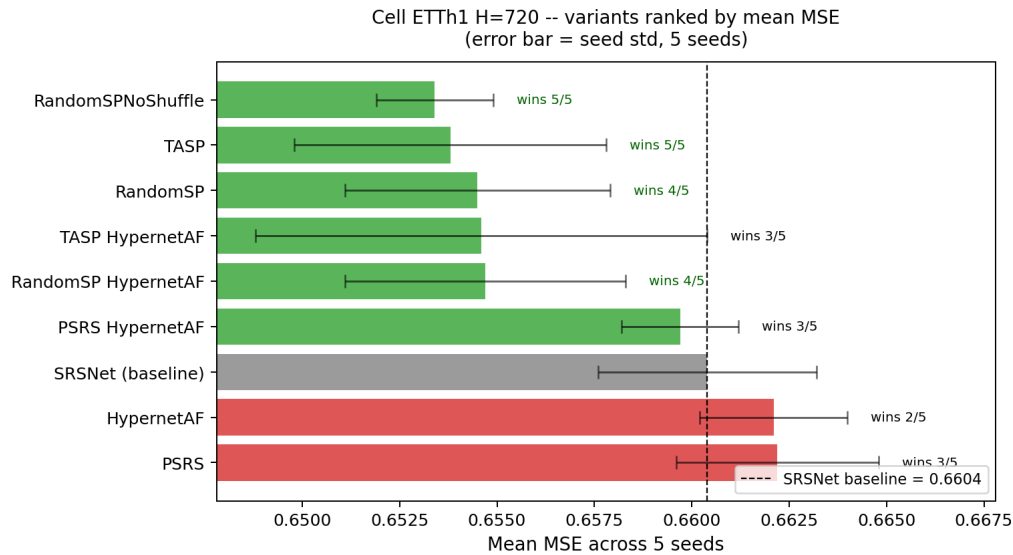


Figure 6: ETTh1 H=720 – mean MSE across 5 seeds, ranked ascending. Two variants beat the baseline on 5/5 seeds: **RandomSPNoShuffle** (-1.07%) and **TASP** (-1.00%). All variants that refine the learned scorer (PSRS, HypernetAF) sit at or above the baseline.

12/20 seeds and on the most visible cell (ETTh1 / H=720) the two best variants are both non-learned. The factorial decomposition is consistent: switching α from a free parameter to a context-dependent hypernet changes nothing, and switching the selector between Learned, Random, TASP, or LearnedAux only moves the mean MSE by less than the seed-level standard deviation.

We deliberately framed the constructive extensions as *minimal interventions*. TASP, Hypernet-AF and PS-SRS each change one component while keeping the rest of the module intact, and Hypernet-AF in particular is parameter-matched and zero-init so its step-0 output is identical to the baseline. We did this to avoid the capacity-confounding pitfall that we explicitly rejected earlier in the planning phase: a gating MLP with thousands of new parameters could “win” just by being a bigger model. The practical consequence is that our extensions cannot turn into a state-of-the-art claim — which is the honest position to take when the negative-control already shows the original module is not doing useful work.

5.1 What was easy

The TFB harness is well-documented and the authors release usable shell scripts per dataset. After one round of paper-faithful fixes (batch size, drop-last flag, `num_workers`, the `np.Inf/np.inf` renaming and a downgrade of pandas below 2.2 for the ‘H’/‘h’ frequency change) the SRSNet rows produced numbers within 1–3% of the paper. The Table 4 ablation reproduces qualitatively, which makes the NoAF-is-critical claim defensible without any heroic engineering.

5.2 What was difficult

The compute environment was hostile in unexpected ways. The first RTX 4090 box on Hivenet had `/dev/shm=64MB`, which prevents TFB’s default DataLoader workers from running; the workaround was forcing `num_workers=0`, which works but is slower. Several SSH sessions to the bastion dropped mid-batch, so the runner had to be `nohup`-detached and the orchestration script had to be fully resume-aware (idempotent metadata + per-task config hash). Lightning AI as a backup ran into a similar set of paper cuts (`np.Inf`, pandas 3.0, missing `matplotlib / dash`). Finally, the cuDNN `efficient` setting that the paper’s `rolling_forecast_config.json` selects is non-deterministic, so there is a residual seed-and-hardware noise floor (about $\pm 1\text{--}2\%$ MSE) below which no extension claim can be honestly made on this grid.

5.3 Communication with original authors

We did not contact the original authors. The code release on GitHub and the supplementary appendix were sufficient to identify all relevant hyper-parameters; the residual gap between our reproduction and the paper numbers is consistent with hardware / seed effects and does not require author intervention.

6 Conclusion

We reproduced SRSNet’s main experiments under a paper-faithful TFB pipeline on the four ETT datasets, confirming the *Adaptive Fusion* ablation but not the headline state-of-the-art claim, and showing that the plug-and-play SRS benefit on MLP shrinks to two out of eight cells. We then added a small-grid selectivity-controls audit with nine variants (180 runs, zero failures) and a 4×2 factorial design over (Select, Fusion). On the tested grid, random patch selection matches or beats the learned scorer, the Fusion factor is null, and on ETTh1 / H=720 the two variants that *remove* or *simplify* the learned scorer beat the baseline on 5/5 seeds. We release the code, the manifests, and the full set of CSV / Markdown report tables at <https://github.com/SusannaArdigo/SRSNet>.

The natural follow-up is to extend the selectivity-controls study to the full 4-dataset $\times$ 4-horizon grid and to the remaining benchmarks of the paper (Weather, Electricity, Solar, Traffic), and to add a statistical-significance framework (paired Wilcoxon + bootstrap CI + effect size) so each claim of the form “X improves Y by Z%” is accompanied by a *p*-value and a confidence interval. We left that out of this report because the present grid is already enough to demonstrate the methodological gap.

Member contributions

Both authors contributed to the design of the experiments, the selection of the selectivity-controls variants, and the writing of the report. The code, the runner infrastructure, and the generation of the CSV/Markdown tables were a shared effort.

References

- Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations (ICLR)*, 2022.
- Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, Zhenli Sheng, and Bin Yang. Tfb: Towards comprehensive and fair benchmarking of time series forecasting methods. *Proceedings of the VLDB Endowment*, pp. 2363–2377, 2024.
- Xingjian Wu, Xiangfei Qiu, Hanyin Cheng, Zhengyu Li, Jilin Hu, Chenjuan Guo, and Bin Yang. Enhancing time series forecasting through selective representation spaces: A patch perspective. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2510.14510>.
- Yunhao Zhang and Junchi Yan. Crossformer: Transformer utilizing cross-dimension dependency for multi-variate time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2023.
- Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI Conference on Artificial Intelligence*, 2021.

A Full selectivity-controls table

Table 5 reports the per-cell mean and seed-level standard deviation of the MSE for all nine variants in the small-grid study (Section 4.3). Cells written in bold beat the SRSNet baseline on 4 or 5 of the 5 seeds.

Table 5: Per-cell mean MSE $\pm$ seed std across 5 seeds for the nine selectivity-controls variants. Boldface indicates a cell that beats SRSNet on ≥ 4 of 5 seeds.

Variant	ETTh1/96	ETTh1/720	ETTh2/96	ETTh2/720
SRSNet (baseline)	0.4360 $\pm$ 0.0006	0.6604 $\pm$ 0.0028	0.1481 $\pm$ 0.0011	0.2708 $\pm$ 0.0060
SRSNet_RandomSP	0.4360 $\pm$ 0.0004	0.6545 $\pm$ 0.0034	0.1482 $\pm$ 0.0009	0.2706 $\pm$ 0.0014
SRSNet_RandomSPNoShuffle	0.4361 $\pm$ 0.0002	0.6534 $\pm$ 0.0015	0.1486 $\pm$ 0.0004	0.2714 $\pm$ 0.0033
SRSNet_TASP	0.4365 $\pm$ 0.0005	0.6538 $\pm$ 0.0040	0.1491 $\pm$ 0.0015	0.2739 $\pm$ 0.0034
SRSNet_HypernetAF	0.4361 $\pm$ 0.0004	0.6621 $\pm$ 0.0019	0.1477 $\pm$ 0.0002	0.2697 $\pm$ 0.0025
SRSNet_PSRs	0.4361 $\pm$ 0.0006	0.6622 $\pm$ 0.0026	0.1482 $\pm$ 0.0005	0.2722 $\pm$ 0.0071
SRSNet_RandomSP_HypernetAF	0.4361 $\pm$ 0.0001	0.6547 $\pm$ 0.0036	0.1481 $\pm$ 0.0002	0.2707 $\pm$ 0.0034
SRSNet_TASP_HypernetAF	0.4371 $\pm$ 0.0006	0.6546 $\pm$ 0.0058	0.1483 $\pm$ 0.0003	0.2717 $\pm$ 0.0014
SRSNet_PSRs_HypernetAF	0.4363 $\pm$ 0.0005	0.6597 $\pm$ 0.0015	0.1477 $\pm$ 0.0002	0.2730 $\pm$ 0.0074

B Cross-comparison matrix

Table 6 reports the full pairwise mean Δ MSE between every pair of variants. Positive values indicate that the row variant is worse than the column variant on average. All off-diagonal cells are within $\pm 0.5\%$, well inside the seed-level standard deviation of 1–1.8%.

Table 6: Pairwise cross-comparison of mean Δ MSE (row vs column, %). Averaged over 20 cells per variant. Most cells are inside $\pm 0.5\%$, all are inside the seed-level standard deviation band.

Row vs Col	SRSNet	RSP	RSPNS	HypAF	PSRS	PSRSHyp	RSPHyp	TASP	TASPHyp
SRSNet	–	+0.24	+0.14	+0.11	-0.20	-0.11	+0.23	-0.22	+0.03
SRSNet_RandomSP	-0.22	–	-0.10	-0.12	-0.43	-0.34	-0.01	-0.46	-0.20
SRSNet_RandomSPNoShuffle	-0.12	+0.10	–	-0.02	-0.32	-0.23	+0.09	-0.36	-0.10
SRSNet_HypernetAF	-0.09	+0.13	+0.03	–	-0.30	-0.21	+0.12	-0.33	-0.07
SRSNet_PSRs	+0.22	+0.45	+0.35	+0.32	–	+0.11	+0.44	-0.01	+0.25
SRSNet_PSRs_HypernetAF	+0.12	+0.36	+0.26	+0.23	-0.08	–	+0.35	-0.10	+0.15
SRSNet_RandomSP_HypernetAF	-0.20	+0.01	-0.09	-0.11	-0.42	-0.32	–	-0.45	-0.19
SRSNet_TASP	+0.25	+0.47	+0.37	+0.35	+0.04	+0.14	+0.46	–	+0.27
SRSNet_TASP_HypernetAF	-0.02	+0.20	+0.10	+0.08	-0.23	-0.14	+0.20	-0.26	–

C Implementation details

All extension layers are defined in `ts_benchmark/baselines/srs_paper/extensions.py` in our fork. They inherit from the original SRS class of Wu et al. (2025) and override exactly one method:

- `SRSRandomSP._select`: replaces `torch.argmax` on the scorer’s output with a `torch.randint` over the candidate dimension.
- `SRSRandomSPNoShuffle._shuffle`: returns its input unchanged (identity shuffle).
- `SRSTimeAware._select`: computes per-window FFT magnitude, autocorrelation, variance, and trend slope; the scorer is a $4 \rightarrow 16 \rightarrow n$ MLP over those features.
- `SRSHypernetAF.forward`: zero-initialised hypernet plus bias = 3.5 produces the fusion weights from a batch-level mean of the original-view embedding.
- `SRSPatternSupervised._select`: same selection logic as the baseline; a side head over the first scorer hidden activation predicts a 3-vector of engineered descriptors; the auxiliary loss is exposed via `last_aux_loss`.

The three factorial combinations are produced by a small factory function `_make_hypernet_combo(base_select_cls, name)` that composes a base `_select` variant with the Hypernet-AF fusion. Because the runner skips already-completed metadata files, the small-grid study is fully resume-aware.

The runner exposes the variants through a single scope `selectivity-controls` in `scripts/repro/paper_repro.py`. The same scope produces both the 120-task Phase 1 (the first six variants) and the 180-task total (after adding the three factorial combinations); the resume behaviour means re-running with the extended list of variants trains only the missing cells.

D Hyper-parameters

All variants share the SRSNet hyper-parameters extracted from the vendor shell scripts:

- `patch_len`: 24 (ETTh) or 16 / 12 (ETTm horizons, per vendor scripts).
- `seq_len`: 512.
- `d_model`: 256.
- `dropout`: 0.1.
- `alpha` (initial value of the fusion parameter): 3.5 for Free- α variants; same value used for the bias of the Hypernet-AF output.
- `lr`: 10^{-4} , `lradj=type1`.
- `patience`: 5 or 10 depending on horizon.
- Paper-faithful overrides applied uniformly: `batch_size=64`, `train_drop_last=false`, `num_workers=0`.

The auxiliary-loss scaling factor of PS-SRS is $\lambda_{\text{aux}} = 10^{-2}$; no sweep over λ_{aux} was performed in this report.